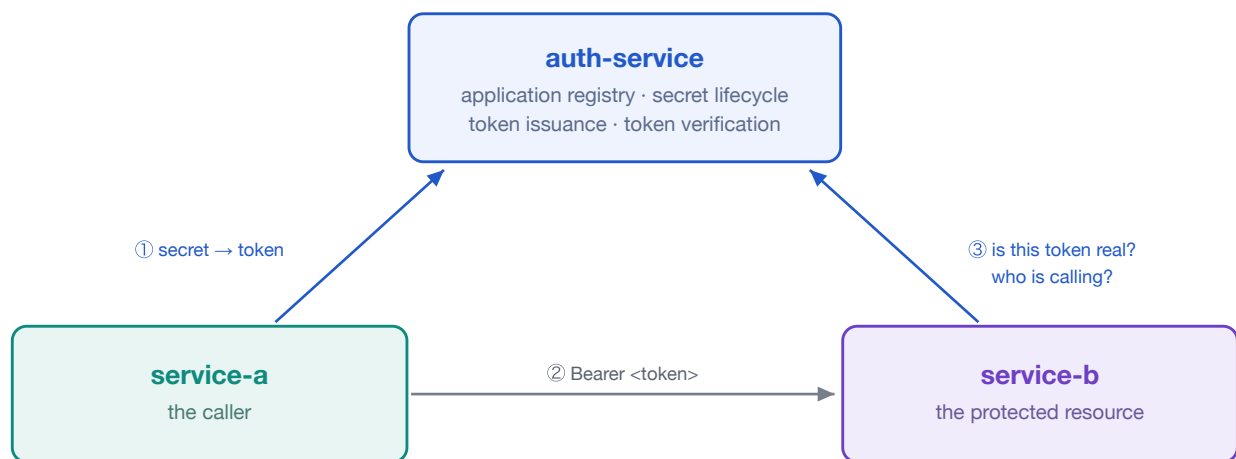


# Service-to-service authentication

How one service proves who it is to another, without a certificate authority and without a long-lived secret ever crossing the wire between them.



## THE DESIGN

- 02 The three services
- 03 One request, end to end

## MANAGING IT

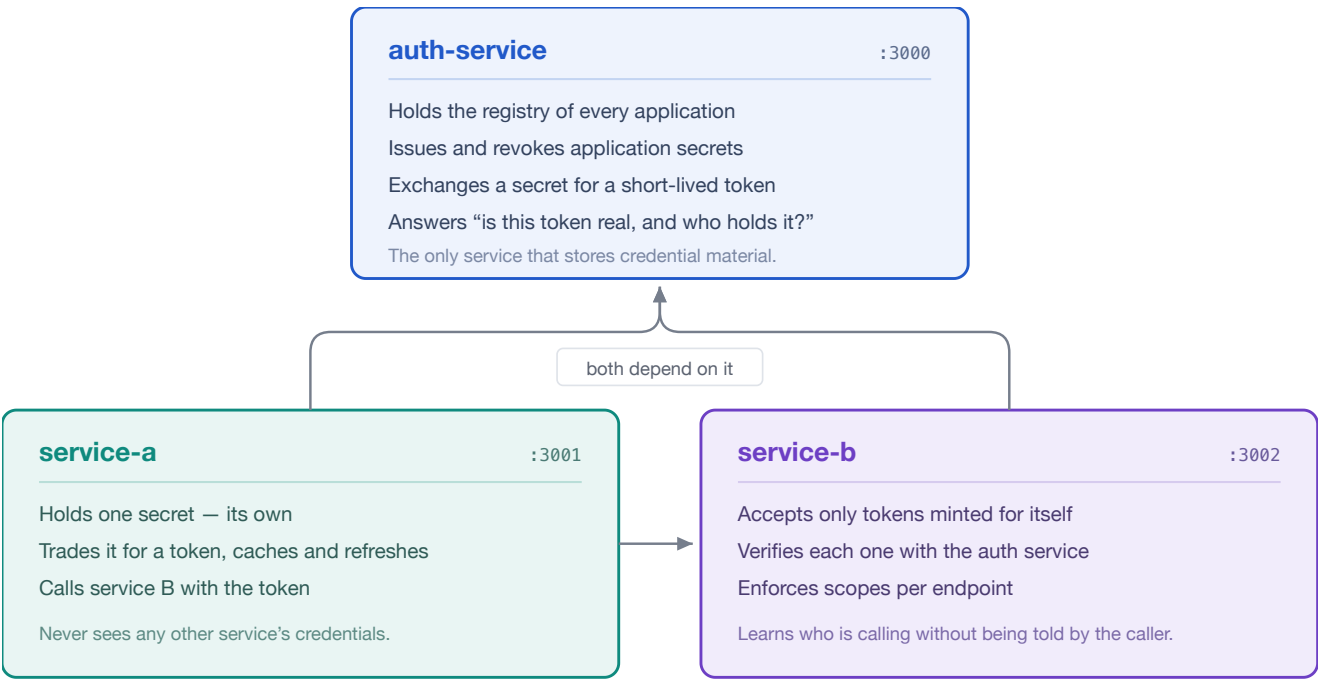
- 04 What the auth service stores
- 05 Adding data to an application
- 06 The management surface
- 07 Secrets and rotation

## OPERATING IT

- 08 When something goes wrong
- 09 In short

# The three services

Each has one job. Service A and service B are ordinary application services; the auth service is the only one that knows anything about credentials.



Service B holds credentials of its own, because verifying a token is itself an authenticated call. Nothing in the system sends a secret on a routine request.

**What travels between A and B**

Only a bearer token: tra\_at\_9f2c... . It expires in ten minutes and is valid at service B and nowhere else.

**What never travels**

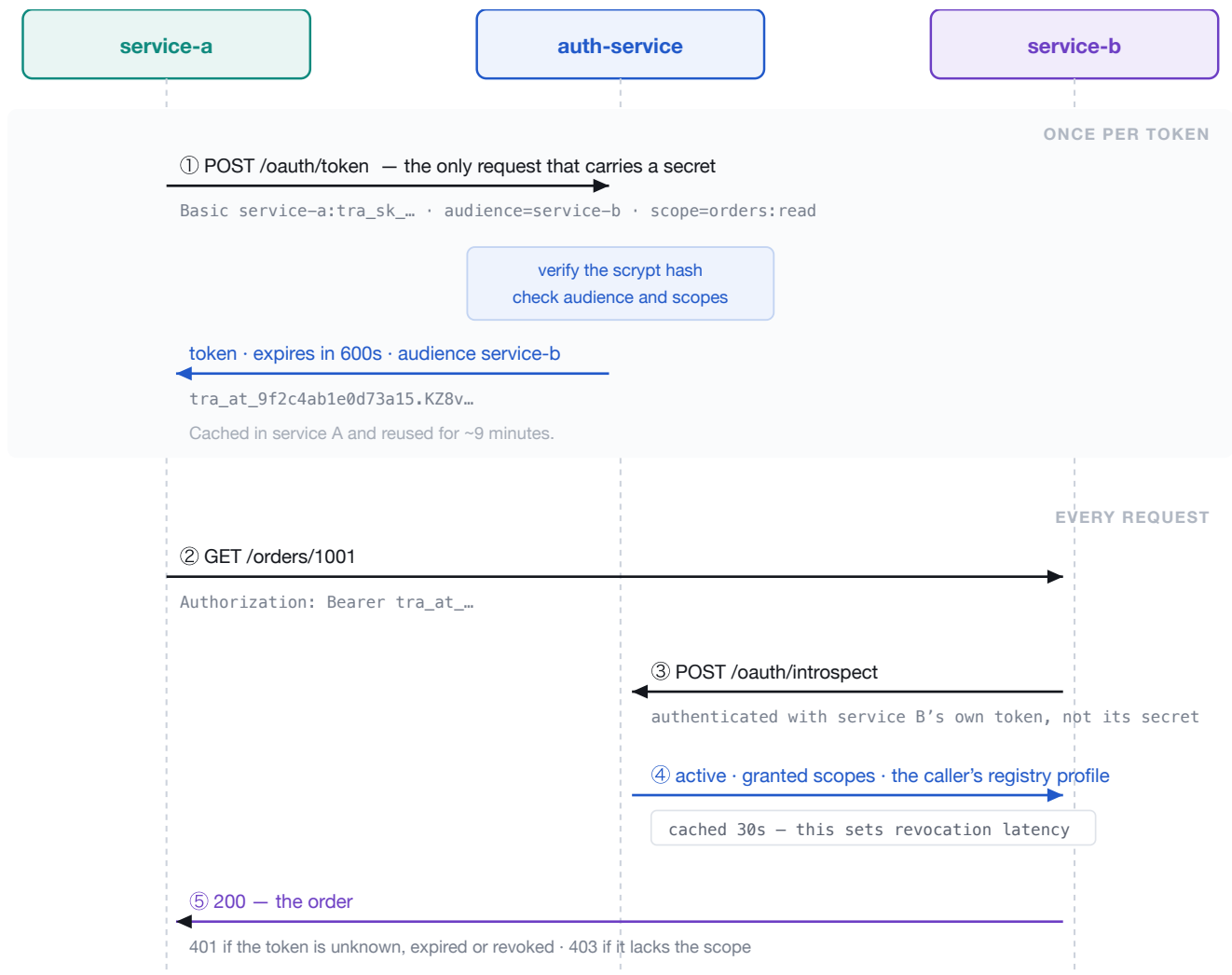
The application secret. It goes to exactly one endpoint, POST /oauth/token , and no further.

| SERVICE      | ROLE                | WHAT IT HOLDS                                                            |
|--------------|---------------------|--------------------------------------------------------------------------|
| auth-service | Issuer and verifier | The registry, every secret hash, the token pepper, the administrator key |
| service-a    | Caller              | Its own client id and secret — nothing else                              |
| service-b    | Resource server     | Its own client id and secret — nothing else                              |

All three are LoopBack 4 applications. A service becomes part of this scheme with one call at start-up and a decorator on each endpoint it wants protected; the verification, caching and refresh logic is shared.

## One request, end to end

Everything below happens on a single call from service A to service B. Steps ① and ④ are cached, so a busy service performs them a few times a minute, not once per request.



If the auth service cannot be reached at step ③, service B answers 503 rather than serving the request. An unavailable verifier is not permission.

### The secret appears once

Step ① is the only place in the system that accepts one. Steps ② to ⑤ carry a token that is useless anywhere but service B.

### Two caches, two dials

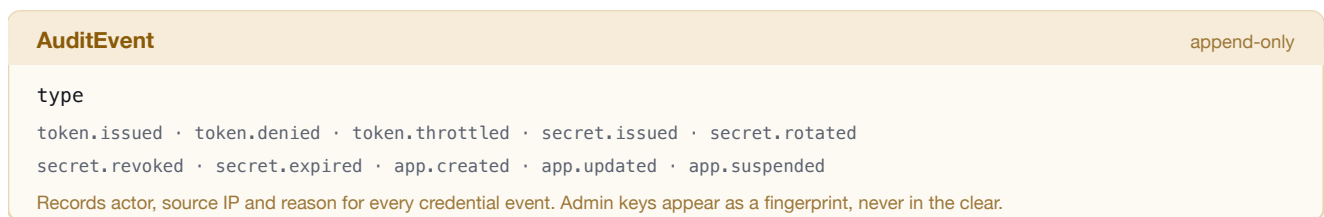
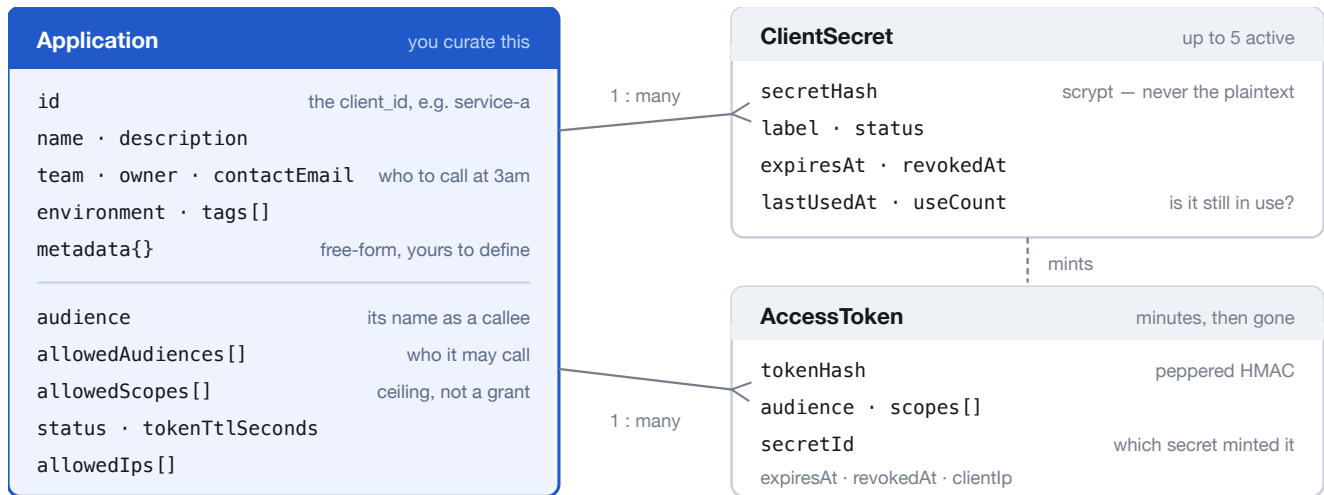
Service A caches the token for its lifetime; service B caches the verification for 30 seconds. The second dial is revocation latency.

### Cost per request

In steady state, ② and ⑤ only. The auth service sees roughly two calls a minute per distinct token, not one per request.

## What the auth service stores

Four records. The **application** is the one you curate — it is where you describe who a caller is. The rest are produced and retired by the system as it runs.



Revoking a secret revokes every token carrying its **secretId** — which is how a leaked credential is contained in one call rather than waiting for tokens to expire.

### You maintain one of these

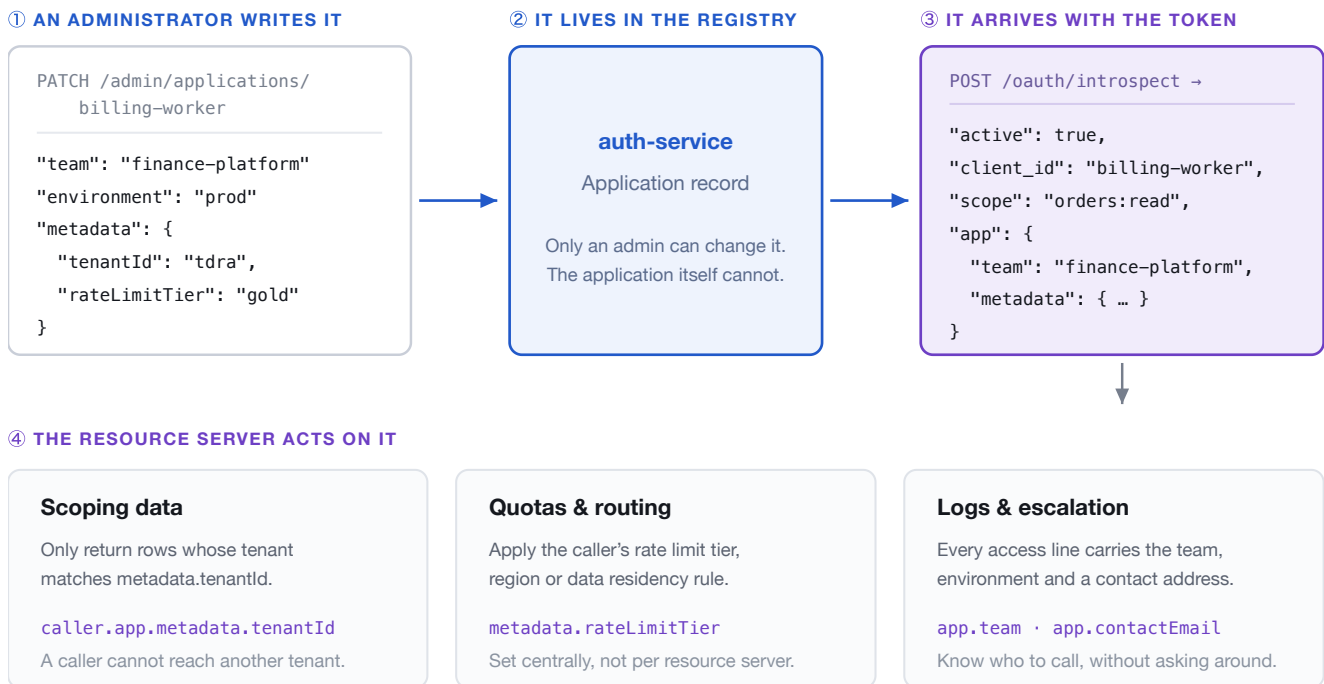
The **application**. Everything else is written by the system as it runs: secrets when you ask for one, tokens on every exchange, audit rows on every event.

### Nothing here is reversible

**secretHash** and **tokenHash** are one-way. There is no endpoint, and no query, that returns a usable credential — including for an administrator.

## Adding data to an application

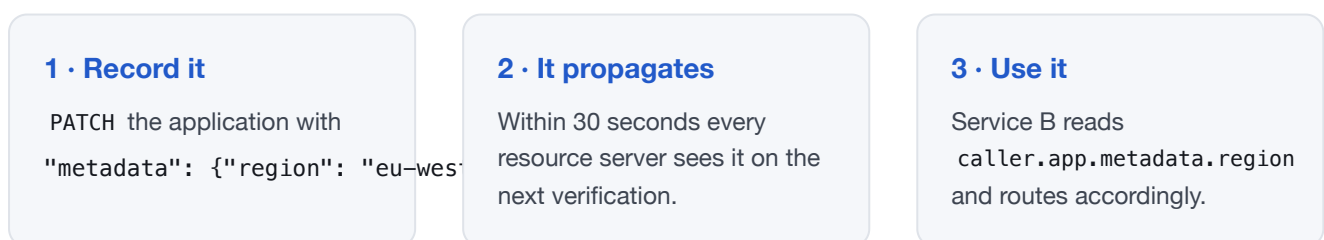
Anything recorded against an application travels with every token it uses, and arrives at the resource server as fact. The caller never handles it, so it cannot forge it.



`metadata` is an open key/value map — it is the extension point. Add whatever your resource servers need to decide about a caller; no schema change is required.

### Worked example — adding a new attribute

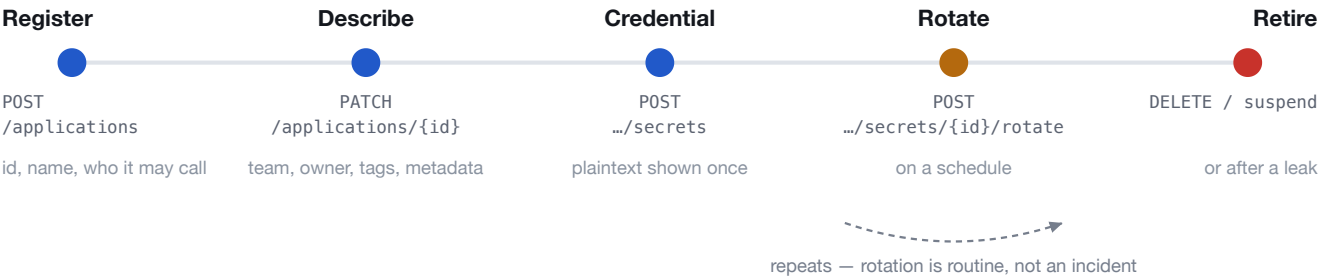
Service B needs to route some callers to a different region. There is no schema change, no migration and no release of the auth service.



The same path carries tenancy, rate limit tiers, cost centres, data residency rules — anything a resource server should know about a caller but must not take the caller's word for.

# The management surface

One API, behind an administrator key. It is a different trust domain from service traffic — it creates the credentials everything else depends on, so it does not ride on the tokens it issues.



| ENDPOINT                                        | WHAT IT IS FOR                                                                                                         |
|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| POST /admin/applications                        | Register a caller. <code>?issueSecret=true</code> returns its first secret in the same response.                       |
| PATCH /admin/applications/{id}                  | <b>Add or change the data a resource server sees</b> — team, owner, contact, environment, tags, metadata, permissions. |
| GET /admin/applications                         | List, filtered by environment, team, tag or status.                                                                    |
| GET /admin/applications/{id}                    | One application with its secrets and live token count.                                                                 |
| POST .../{id}/suspend<br>POST .../{id}/activate | Reversible kill switch. Stops issuance and revokes outstanding tokens; keeps the history.                              |
| POST .../secrets                                | Issue a credential. Plaintext returned once, stored only as a hash.                                                    |
| GET .../secrets                                 | Status, expiry, last use — metadata only, never a hash.                                                                |
| POST .../secrets/{id}/rotate                    | Issue a replacement and put the old one on a countdown.                                                                |
| PATCH .../secrets/{id}/expiry                   | Shorten a suspect credential, or extend one about to lapse.                                                            |
| DELETE .../secrets/{id}                         | Revoke now, along with every token the secret minted.                                                                  |
| GET /admin/audit                                | Every credential event, filtered by application, type or outcome.                                                      |

An importable Postman collection covers all of this — 42 requests, including the failure cases, each asserting its own expected outcome.



## When something goes wrong

Four levers, in increasing blast radius. Reach for the narrowest one that covers the problem.

### One token

POST /oauth/revoke

### One secret — and every token it minted

DELETE /admin/applications/{app}/secrets/{id}

### One application — all of its secrets and tokens at once

POST /admin/applications/{app}/suspend · reversible

### Every token in the fleet

Rotate TOKEN\_PEPPER and restart · services recover on their own within one token lifetime · no redeploy

Latency: immediate at the auth service, and up to the verification cache window — 30 seconds by default — at each resource server.

## Credentials are held separately

Each service reads only its own configuration. One compromised service does not become every service.

### auth-service

TOKEN\_PEPPER  
ADMIN\_API\_KEY  
every secret hash

The one service that must be protected as if it were a database of credentials — because it is.

### service-a

CLIENT\_ID  
CLIENT\_SECRET  
— and nothing else

Cannot read service B's secret, the pepper, or the admin key.

### service-b

CLIENT\_ID  
CLIENT\_SECRET  
— and nothing else

Verifies tokens, but cannot issue or forge one.

Enforced locally by a separate .env per service; in production, by your secret store's access policy.

## In short

### The mechanism

- A secret is exchanged for a short-lived token at one endpoint.
- The token is opaque and bound to a single recipient.
- The recipient verifies it with the issuer, and caches the answer.
- No certificates, no signing keys, no key distribution.

### The management model

- Applications are registered and described centrally.
- That description reaches every resource server as fact.
- Secrets are issued, rotated and revoked without a redeploy.
- Every credential event is recorded.

## Properties worth restating

| PROPERTY                                             | HOW IT IS ACHIEVED                                                                                                      |
|------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| A compromised service B cannot impersonate service A | B only ever receives a token minted for B.                                                                              |
| A leaked credential is contained in one call         | Revoking a secret revokes every token it minted.                                                                        |
| A database leak yields nothing usable                | Secrets are script hashes; tokens are HMACs under a pepper held outside the database.                                   |
| Probing reveals nothing                              | Wrong secret, unknown application and malformed credential all return one indistinguishable response, in the same time. |
| An outage does not become an opening                 | If the auth service cannot be reached, resource servers refuse the request.                                             |
| Callers cannot lie about themselves                  | Their profile comes from the registry at verification time, never from the request.                                     |

## Before production

- TLS on every hop — the secret crosses the network once.
- Move storage from the in-memory connector to PostgreSQL.
- Replace the shared administrator key with per-operator identity behind SSO.
- Run more than one replica: every call depends on this service.
- Back up the token pepper; losing it forces a fleet-wide re-authentication.
- Alert on denied and throttled token requests, and on secrets nearing expiry.

Further detail: [README.md](#) for the design rationale and full configuration, [docs/operations.md](#) for the runbooks — onboarding a service, scheduled rotation, and responding to a leaked secret.